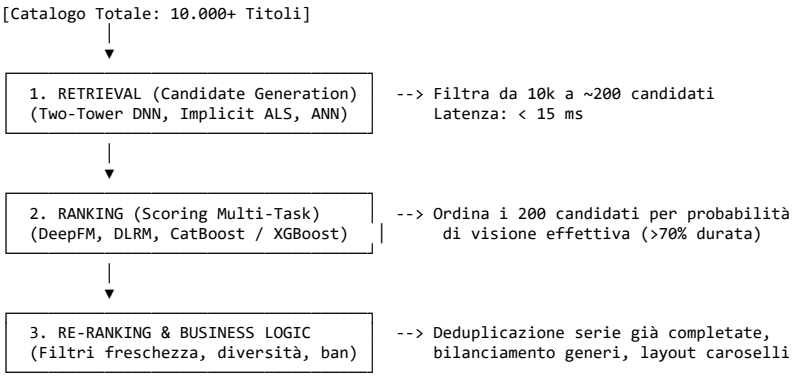
Algoritmi Open Source per Raccomandazione Film

https://gemini.google.com/app/801ba27d35bb9d5f

User prompt: Ricerca algoritmi/codice open source: migliori sistema di raccomandazioni film/serie

Response: I moderni motori di raccomandazione per piattaforme di streaming (come Netflix, Prime Video o Disney+) non utilizzano un singolo modello isolato, ma una pipeline a tre stadi: **Candidate Generation (Retrieval)**, **Ranking (Scoring contestuale)** e **Re-ranking / Diversification**.

1. Architettura di Riferimento (Pattern Netflix / YouTube)



- **Feedback Implicito vs Esplicito:** Nello streaming i voti (stelle o pollici) sono rari (<1%). I dati primari sono le interazioni implicite: completamento del video (es. >80% = positivo, <5 min = skip), click sulla locandina, aggiunta alla watchlist e tempo di permanenza sulla scheda.
- **Session & Sequence Awareness:** La sequenza temporale conta. Chi finisce una saga si aspetta il capitolo successivo o spin-off dello stesso franchise, mentre i gusti cambiano tra sessioni pomeridiane e serali.

2. I Migliori Algoritmi

Famiglia	Algoritmi Chiave	Perché eccelle con Film e Serie
Matrix Factorization (Implicita)	iALS (Implicit ALS), BPR-MF	Estremamente veloce ed efficiente su matrici sparse con milioni di visualizzazioni. Perfetto come baseline di retrieval.
Two-Tower Neural Networks	Dual Encoders (User Tower + Item Tower)	Mappa profili utente e metadati video nello stesso spazio vettoriale. Permette lookup sub-millisecondo con vector database (FAISS, Qdrant).
Sequential / Transformer-based	SASRec, BERT4Rec, Transformers4Rec	Cattura la dinamica temporale: predice il "prossimo episodio/film" basandosi sulla sequenza cronologica esatta degli ultimi titoli visti.
Graph Neural Networks (GNN)	LightGCN, PinSage	Modella relazioni complesse tra nodi: Utente ↔ Film ↔ Regista ↔ Cast ↔ Genere. Ottimo per spiegabilità ("Perché hai visto X con Y").
Hybrid Content-Based (LLM)	Sentence-BERT / Text Embeddings + Cosine Sim	Risolve il problema del **Cold Start**: quando esce un film appena caricato nel catalogo (zero visualizzazioni), ne calcola la somiglianza tramite sinossi, tag e cast.

3. Le Migliori Librerie e Repository Open Source

1. Microsoft Recommenders (`microsoft/recommenders` / Linux Foundation)

Il repository open-source più strutturato per ambienti di produzione. Contiene decine di notebook end-to-end benchmarkati direttamente su dataset cinematografici (MovieLens 100k, 1M, 20M).

- **Supporta:** SAR, NCF (Neural Collaborative Filtering), LightGCN, VAE, DKN, xDeepFM.
- **GitHub:** `[github.com/recommenders-team/recommenders](https://github.com/recommenders-team/recommenders)`

2. RecBole (`RUCAIBox/RecBole`)

Il framework PyTorch accademico e industriale più completo. Include oltre 80 algoritmi nativi, dai classici di fattorizzazione fino ai modelli Transformer (`RecBole-TRM`) e Graph (`RecBole-GNN`).

- **Ideale per:** Benchmark scientifico comparativo e prototipazione rapida con file di configurazione YAML.
- **GitHub:** `[github.com/RUCAIBox/RecBole](https://github.com/RUCAIBox/RecBole)`

3. Implicit (`benfred/implicit`)

Libreria C++/Cython con binding Python ultra-ottimizzata per CPU multithread e GPU. È lo standard di mercato per matrici di visualizzazione/skip basate su Implicit Feedback.

- **Algoritmi principali:** Alternating Least Squares (ALS), Bayesian Personalized Ranking (BPR), Logistic Matrix Factorization.
- **GitHub:** `[github.com/benfred/implicit](https://github.com/benfred/implicit)`

4. NVIDIA Merlin (`NVIDIA-Merlin/Merlin`)

Suite end-to-end accelerata su GPU per cataloghi su larga scala.

- **Componenti:** `NVTabular` (feature engineering terabyte-scale su GPU), `Transformers4Rec` (session-based con HuggingFace), `Triton Inference Server` per il serving in tempo reale.
- **GitHub:** `[github.com/NVIDIA-Merlin/Merlin](https://github.com/NVIDIA-Merlin/Merlin)`

5. Scikit-Surprise (`surpriselib`)

Libreria scikit-learn compatibile per dataset a rating esplicito.

- **Ideale per:** Piccoli progetti, Proof-of-Concept, implementazioni classiche di SVD / SVD++ / k-NN.
- **GitHub:** `[github.com/NicolasHug/Surprise](https://github.com/NicolasHug/Surprise)`

4. Implementazioni Pratiche

Pipeline A: Collaborative Filtering Implicito con `implicit` (ALS)

Ottimizzato per log di streaming: l'utente non assegna voti, ma accumula ore di visione o visualizzazioni confermate.

```
import implicit
import numpy as np
from scipy.sparse import csr_matrix

# 1. Creazione della matrice d'interazione (Utenti x Titoli)
# Valore = segnale di confidenza (es. % di completamento o minuti visti)
rows = np.array([0, 0, 1, 1, 2, 2, 3])  # User IDs
cols = np.array([101, 102, 101, 103, 102, 104, 103])  # Movie IDs
confidence = np.array([5.0, 1.0, 4.0, 3.0, 2.0, 5.0, 4.0])

user_item_matrix = csr_matrix((confidence, (rows, cols)), shape=(4, 105))

# 2. Inizializzazione e training del modello ALS
# factors = dimensionalità dei vettori latenti
model = implicit.als.AlternatingLeastSquares(
    factors=64,
    regularization=0.05,
    iterations=20,
    random_state=42
)
model.fit(user_item_matrix)

# 3. Raccomandazioni personalizzate per l'utente 0 (Top 3)
user_id = 0
ids, scores = model.recommend(
    user_id,
    user_item_matrix[user_id],
    N=3,
    filter_already_liked_items=True
)
print("Film raccomandati per Utente 0:", ids)

# 4. Item-to-Item: "Titoli simili a questo" (per carosello orizzontale)
target_movie_id = 101
similar_ids, sim_scores = model.similar_items(target_movie_id, N=3)
print(f"Film simili a {target_movie_id}:", similar_ids)
```

Pipeline B: Content-Based con Text Embeddings (Risoluzione Cold Start)

Quando viene inserito un film o una serie TV priva di cronologia interazioni, si estraggono i vettori semantici da sinossi, generi e attori per la ricerca vettoriale con FAISS.

```
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer

# 1. Modello per estrarre vettori semantici compatti
model = SentenceTransformer('all-MiniLM-L6-v2')

# 2. Catalogo descrittivo (Metadata enrichment)
movies = [
    {"id": 1, "title": "Inception", "text": "Sci-Fi Thriller. Dom Cobb ruba segreti aziendali attraverso la tecnologia di condivisione dei sogni."},
    {"id": 2, "title": "Interstellar", "text": "Sci-Fi Avventura. Un gruppo di esploratori viaggia attraverso un wormhole nello spazio profondo."},
    {"id": 3, "title": "The Hangover", "text": "Commedia. Tre amici si svegliano a Las Vegas dopo una notte brava senza ricordare nulla."},
]

corpus = [m["text"] for m in movies]
embeddings = model.encode(corpus, convert_to_numpy=True, normalize_embeddings=True)

# 3. Indicizzazione vettoriale con FAISS
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)  # Inner Product su vettori normalizzati = Cosine Similarity
index.add(embeddings)

# 4. Query di ricerca o nuovo film simile (Cold Start)
query_film = "Missione spaziale per salvare l'umanità tra pianeti lontani"
query_vector = model.encode([query_film], convert_to_numpy=True, normalize_embeddings=True)

k = 2
distances, indices = index.search(query_vector, k)

print("Risultati più affini:")
for idx, score in zip(indices[0], distances[0]):
    print(f"- {movies[idx]['title']} (Score di somiglianza: {score:.4f})")
```

5. Dataset Standard per il Benchmark

- **MovieLens (GroupLens Research):** Il golden standard per testare algoritmi. Disponibile nelle varianti `ml-100k` (didattico), `ml-1m`, `ml-20m` e `ml-32m` (con milioni di rating e tag).
- **The Movies Dataset (TMDB / Kaggle):** 45.000+ film con metadati completi (budget, cast, troupe, locandine, trame).

- **Netflix Prize Dataset:** Il dataset storico che ha dato vita alla moderna competizione sui motori di raccomandazione (100 milioni di rating).

Sources

1. List of Recommender Systems - GitHub
2. NVIDIA-Merlin - GitHub
3. NVIDIA Merlin - GitHub
4. Surprise · A Python scikit for recommender systems.

User prompt: Non mostrarmi codice, dimmi i migliori disponibili, avendo a disposizione il dump di tmdb

Response: Il dump ufficiale di TMDB ha una caratteristica fondamentale: **contiene metadati ricchi (sinossi, cast, registi, keyword tematiche, generi, saghe/franchise e popolarità globale), ma non include le cronologie dei singoli utenti.**

Senza interazioni utente individuali, non è possibile applicare il Collaborative Filtering puro (come ALS o Two-Tower). Con il solo dump di TMDB, l'eccellenza si ottiene sfruttando il **Content-Based semantico**, i **Knowledge Graph** e la **Similarità Ibrida tra Entità**.

1. I Migliori Algoritmi da Applicare al Dump TMDB

Dense Vector Retrieval (Embedding Semantici)

- **Come funziona:** Converte la sinossi (*overview*), il tono e le parole chiave tematiche in vettori densi nello spazio semantico.
- **Perché eccelle con TMDB:** Comprende il "mood", i tropi narrativi e l'atmosfera. Se un utente cerca o apprezza un thriller psicologico con colpo di scena nello spazio, l'algoritmo non cerca solo parole coincidenti, ma trova film con premesse narrative analoghe anche se usano vocaboli differenti.
- **Modelli di riferimento:** Famiglia BGE, E5, o Sentence-Transformers multilingue.

Graph Embeddings (FastRP, Node2Vec)

- **Come funziona:** Modella il dump di TMDB come una rete di nodi (*Film*, *Attore*, *Regista*, *Genere*, *Casa di produzione*, *Collezione/Franchise*) ed estrae vettori che catturano la vicinanza topologica.
- **Perché eccelle con TMDB:** TMDB è un grafo naturale. Questo approccio riconosce schemi ricorrenti (es. l'accoppiata regista-direttore della fotografia, attori feticcio, saghe collegate) senza bisogno di scrivere centinaia di regole manuali.

Weighted Multi-Attribute Similarity (Cosine / BM25 pesato)

- **Come funziona:** Calcola la somiglianza pesando selettivamente i vari campi di TMDB: un peso alto per il regista e il franchise, medio per i primi tre attori del cast e le keyword, basso per la sinossi generale.
- **Perché eccelle con TMDB:** È l'approccio più interpretabile e controllabile. Evita l'effetto "allucinazione" dei vettori puri e garantisce che film dello stesso franchise o con lo stesso autore salgano immediatamente in cima alla lista dei titoli correlati.

Punteggio Bayesiano Ponderato (Ranking per Popolarità e Voto)

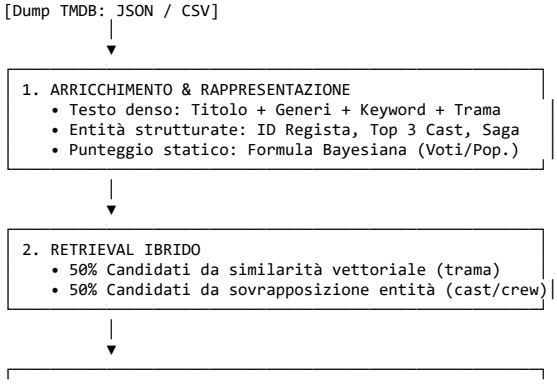
- **Come funziona:** Formula statistica usata originariamente da IMDb che combina `vote_average` e `vote_count` con la media globale del catalogo.
- **Perché serve:** Nei dump di TMDB, film di nicchia con un solo voto a 10/10 rischiano di scavalcare capolavori con 20.000 voti e media 8.5. Il rating bayesiano normalizza la qualità percepita e funge da moltiplicatore per ordinare i risultati simili.

2. I Migliori Motori e Strumenti Open Source

Motore / Piattaforma	Punto di Forza con TMDB	Casi d'Uso Tipici
Qdrant	Vector database ad alte prestazioni con filtraggio su metadati (*payload filtering*).	Generazione di caroselli "Film simili a questo" basati su trama, filtrando al volo per genere o anno.
Vespa.ai	Il motore ibrido open source più avanzato al mondo per ranking e raccomandazioni.	Unisce ricerca semantica vettoriale, BM25 sui testi e modelli di machine learning per lo scoring finale in un'unica istanza.
Neo4j (Graph Data Science)	Database a grafo con algoritmi integrati di similarità e community detection.	Esplorazione relazionale profonda: "Perché ti è piaciuto questo film dello stesso regista con questo cast".
Meilisearch / Typesense	Motori di ricerca istantanea focalizzati sulla tolleranza agli errori e navigazione a faccette.	Discovery rapida con filtri multi-parametro (attore + genere + range di anni) e ordinamento per popolarità.

3. L'Architettura Consigliata per Sfruttare TMDB

La strategia più efficace per trasformare il dump di TMDB in un motore di raccomandazione completo si articola su tre livelli:



```
3. SCORING & DIVERSIFICAZIONE
   • Punteggio = Similarità × log(Popolarità bayesiana)
   • Re-ranking: massimo 2 film della stessa saga
```

4. Come Aggiungere la Personalizzazione Utente

Se in futuro vorrai raccomandare titoli in base ai gusti personali di ciascun utente e non solo per affinità tra film:

- **Mappatura con MovieLens:** Il dataset accademico MovieLens fornisce nativamente il file di corrispondenza con gli ID di TMDB (`tmdbId`). Puoi scaricare le matrici di voti di MovieLens, addestrare algoritmi collaborativi all'avanguardia (come **EASE** o **LightGCN**) e arricchire l'interfaccia finale con le locandine, i cast e i dettagli estratti dal tuo dump TMDB.
- **Profilazione Utente Vettoriale (Centroid Vector):** Senza installare complessi sistemi di machine learning, puoi calcolare il vettore di un utente facendo la media dei vettori dei film a cui ha messo "Mi piace", per poi interrogare il database vettoriale alla ricerca dei film del catalogo più vicini a quel centroide.

User prompt: Aspetta, non voglio embeddare tutte le sinossi ma solo le keyword. Inoltre, per gli user ho lo storico delle visioni. Quali degli algoritmi già esistenti è il migliore nel mio caso?

Response: Avendo a disposizione lo **storico delle visioni** (feedback implicito collaborativo) e le sole **keyword di TMDB** (attributi discreti e semantici dei film), il problema si trasforma nel classico scenario di **Hybrid Collaborative Filtering con Item Side-Information**.

Il verdetto dipende dal livello di complessità dell'infrastruttura, ma un modello spicca su tutti per questo caso specifico.

1. Il Vincitore Assoluto: Two-Tower Neural Network (Retrieval Tower)

È l'architettura di riferimento adottata dai principali servizi di streaming (YouTube, Netflix, Spotify) quando si combinano cronologie utente e metadati mirati come le keyword.

- **Come modella i tuoi dati:**
  - **User Tower:** Prende in input la sequenza o l'insieme degli ID dei film già visti dall'utente e ne genera una rappresentazione vettoriale densa dinamica.
  - **Item Tower:** Invece di usare una sinossi lunga, prende il **vettore multi-hot o l'aggregazione delle sole keyword TMDB** del titolo (es. *time travel*, *heist*, *dystopia*), proiettandole nello stesso spazio vettoriale dell'utente.
- **Perché è il migliore per te:**
  - Gestisce contemporaneamente l'affinità collaborativa ("utenti che guardano X guardano anche Y") e la coerenza tematica data dalle keyword.
  - Risolve il **Cold Start dei nuovi film**: se entra in catalogo un titolo con zero visualizzazioni, la Item Tower legge le sue keyword e lo posiziona istantaneamente vicino agli utenti che amano quei temi.
  - In fase di serving è rapidissimo: calcoli il prodotto scalare tra il vettore utente e i vettori dei film memorizzati su un vector search engine.

2. Il Migliore per Semplicità e Risorse Contenute: LightFM

Se non vuoi addestrare reti neurali deep complesse o gestire pipeline pesanti, **LightFM** (con loss function *WARP* o *BPR*) è la scelta più collaudata ed efficiente.

- **Come modella i tuoi dati:**
  - Estende la classica fattorizzazione di matrice (Matrix Factorization): un film non è rappresentato da un ID isolato, ma dalla **somma dei vettori delle sue feature** (le tue keyword TMDB).
  - L'utente viene descritto attraverso l'interazione tra i suoi gusti latenti e le keyword dei titoli che ha guardato.
- **Perché considerarlo:**
  - Estremamente leggero da addestrare anche su CPU.
  - Se due film condividono keyword rare in TMDB (es. *cyberpunk* + *neo-noir*), il modello trasferisce la preferenza dell'utente da un film all'altro anche con pochissime visualizzazioni registrate.

3. Il Migliore per Sessioni Sequenziali: S3-Rec / SASRec con Attributi

Se per la tua piattaforma l'**ordine temporale delle visioni** è decisivo (es. "cosa consigliare subito dopo aver finito una specifica trilogia o serie"):

- **Come modella i tuoi dati:**
  - Utilizza blocchi di **Self-Attention (Transformer)** sulla sequenza cronologica dei film visti.
  - **S3-Rec (Self-Supervised Sequential Recommendation):** È una variante progettata appositamente per sfruttare gli attributi (keyword). Utilizza compiti auto-supervisionati per correlare la sequenza temporale dei click/visioni con le keyword associate ai film di ciascun passaggio.
- **Perché considerarlo:** Batte qualsiasi modello matriciale nel predire il *prossimo* contenuto esatto all'interno della stessa sessione serale di visione.

4. Il Migliore Basato su Grafi: KGAT / LightGCN con Grafo Eterogeneo

Se vuoi sfruttare la struttura di rete naturale tra utenti, film e keyword:

- **Come modella i tuoi dati:**
  - Costruisce un grafo eterogeneo tripartito: `[Utente] ---ha visto---> [Film] ---ha keyword---> [Tag TMDB]`.
  - **KGAT (Knowledge Graph Attention Network):** Applica meccanismi di attenzione per capire *quale specifica keyword* ha guidato la visione di un utente rispetto a un'altra (es. ha visto *Inception* per la keyword *mind-bending* o per la keyword *crime*?).
- **Perché considerarlo:** Massima capacità di spiegare il perché della raccomandazione (*explainability* diretta nell'interfaccia utente).

Sintesi e Verdetto Operativo

Algoritmo	Complessità	Scalabilità	Gestione Cold-Start con Keyword	Ideale per
Two-Tower DNN *(Scelta Top)*	Media-Alta	Eccellente (Milioni di item/user)	Molto Alta	Servizi di streaming con catalogo attivo e ricerca vettoriale sub-secondo.
LightFM (WARP)	Bassa	Buona (fino a medie scale)	Alta	Prototipi veloci, startup, server con sole CPU e manutenzione minima.
S3-Rec / SASRec	Alta	Buona	Media-Alta	Piattaforme focalizzate sulla sessione corrente e sull'effetto sequenziale.
KGAT (GNN)	Molto Alta	Complessa da scalare	Alta	Sistemi che richiedono spiegazioni semantiche dettagliate dei suggerimenti.

Se l'obiettivo è un sistema moderno, scalabile e reattivo, la combinazione più efficace è la **Two-Tower DNN**, addestrando l'Item Tower direttamente con gli embedding combinati delle keyword TMDB.

User prompt: Non ho lo storico di altri utenti ma solo di un singolo utente, quindi l'affinità collaborativa mi manca (a meno di trovare un dataset utile allo scopo)

Response: Senza i dati di altri utenti viene meno il presupposto del Collaborative Filtering tradizionale (nessuna matrice condivisa tra utenti). Il problema ricade quindi in due opzioni chiare:

1. **Rimanere con i tuoi soli dati:** algoritmi di **Single-User Content-Based Profiling** e **Graph Traversal**.
2. **Sfruttare un dataset esterno:** mappare TMDB su **MovieLens** tramite gli ID nativi e proiettare il tuo utente nello spazio collaborativo (*Folding-in*).

Opzione 1: Usare solo i tuoi dati (Single-User)

In questo scenario, il modello deve costruire un **profilo di preferenze dinamico** del singolo utente basandosi unicamente sulle keyword TMDB dei titoli che ha guardato.

1. Il Migliore: Dynamic User Centroid con Decadimento Temporale (Algoritmo di Rocchio)

È l'algoritmo classico e più robusto per profilare un singolo utente:

- **Rappresentazione del film:** Ogni titolo è rappresentato da un vettore compatto ottenuto aggregando (es. mean pooling o TF-IDF pesato) solo le rappresentazioni delle sue keyword TMDB.
- **Costruzione del profilo utente:** Il profilo dell'utente è un **vettore centroide** calcolato come la media pesata dei vettori dei film visti.
- **I punti di forza:**
  - **Decadimento temporale (Time Decay):** I film visti ieri pesano di più nel centroide rispetto a quelli visti un anno fa, rispecchiando i gusti attuali.
  - **Feedback negativo:** Se l'utente ha abbandonato un film dopo 10 minuti, le sue keyword vengono sottratte dal centroide (formula di Rocchio).
  - **Matching istantaneo:** I film del catalogo vengono ordinati per somiglianza del coseno rispetto al vettore profilo dell'utente.

2. Personalized PageRank (PPR) su Grafo Bipartito

Se non vuoi trasformare le keyword in vettori densi e preferisci sfruttare le connessioni dirette:

- **Come modella i dati:** Crei una rete semplice con due tipi di nodi: **Film** e **Keyword TMDB**. Se il film A ha le keyword *heist* e *revenge*, esistono archi tra A e quelle due keyword.
- **Come funziona:** Esegui una camminata casuale con riavvio (*Random Walk with Restart*) partendo esclusivamente dai nodi dei film presenti nella cronologia dell'utente.
- **Perché è eccellente:** Calcola la probabilità con cui il cammino casuale approda a film del catalogo non ancora visti. Più keyword rare e specifiche condividono i film candidati con la cronologia dell'utente, maggiore sarà il loro punteggio. È intrinsecamente immune all'overfitting e cattura relazioni multi-step (es. keyword che spesso co-occorrono).

3. BM25 / TF-IDF Pesato sulle Keyword (User-as-Query)

L'approccio più semplice e deterministico da implementare su un motore di ricerca:

- **Come funziona:** La cronologia dell'utente diventa una "mega-query" di keyword. Le keyword dei film visti più volte assumono un peso (frequenza) maggiore.
- **Perché funziona:** La componente IDF (Inverse Document Frequency) penalizza automaticamente le keyword TMDB troppo generiche o frequenti (come *based on novel* o *friendship*) e premia le keyword di nicchia e identitarie (come *time loop* o *cyberpunk*), garantendo raccomandazioni molto precise.

Opzione 2: Integrare un Dataset Esterno (Il "Ponte" MovieLens)

Se desideri l'effetto "chi ha visto questo ha visto anche...", puoi colmare la mancanza di altri utenti integrando il dataset accademico **MovieLens** (versione `ml-25m` o `ml-32m`).

Perché funziona subito con TMDB

MovieLens include nativamente il file `links.csv`, che contiene la mappatura 1:1:

$$\text{movieId (MovieLens)} \longleftrightarrow \text{tmdbId (TMDB)}$$

L'algoritmo migliore: EASE con Proiezione ("Folding-In")

EASE (Embarrassingly Shallow Autoencoders) è considerato uno dei migliori algoritmi collaborativi lineari:

- **Addestramento offline:** Addestri EASE sui milioni di utenti di MovieLens per calcolare una matrice simmetrica di transizione tra film (B).
- **Proiezione del singolo utente (*Folding-In*):** Non hai bisogno di riaddestrare il modello. Prendi il vettore binario del tuo singolo utente (1 sui film visti, 0 sugli altri) e lo moltiplichi direttamente per la matrice B:

$$\text{Punteggi} = \text{Cronologia_Utente} \times B$$

- **Iniezione delle Keyword TMDB:** Ottenuti i punteggi collaborativi globali, applichi un moltiplicatore basato sull'affinità con le keyword TMDB del tuo utente, ottenendo una

raccomandazione ibrida completa.

Verdetto: Quale scegliere?

- Se vuoi un sistema **autonomo, leggero e immediato da implementare** senza scaricare dataset da decine di gigabyte:

  👉 **Dynamic User Centroid (Rocchio) sulle keyword con decadimento temporale.** È trasparente, aggiornabile in tempo reale a ogni nuova visione e non soffre la mancanza di altri utenti.

- Se vuoi la **vera logica di affinità collaborativa** in stile streaming commerciale:

  👉 Scarica **MovieLens 25M/32M**, usa `links.csv` per allineare gli ID al tuo dump TMDB e applica **EASE via Folding-In**, usando poi le keyword TMDB come filtro di re-ranking.

User prompt: Interessante movielens, potrei crearmi un filtro collaborativo senza avere dati storici dei miei user. Spiegami meglio

Response: L'idea alla base di questa strategia è **trasferire l'intelligenza collettiva** di centinaia di migliaia di persone esterne sul tuo catalogo, pre-calcolando una mappa universale di affinità tra film. Invece di dipendere dai tuoi utenti per capire cosa è correlato a cosa, usi MovieLens come "popolazione di addestramento".

1. Il Ponte: Come collegare MovieLens a TMDB

Il dataset MovieLens (nello specifico le versioni `ml-25m` o `ml-32m`) contiene decine di milioni di voti assegnati da oltre 160.000 utenti a circa 60.000–80.000 film.

La chiave di volta tecnica è un singolo file fornito nativamente nel dataset: **`links.csv`**.

`movieId` (MovieLens)	`imdbId`	`tmdbId` (TMDB)
`1`	`0114709`	`862` *(Toy Story)*
`2`	`0113497`	`8844` *(Jumanji)*
`296`	`0110912`	`680` *(Pulp Fiction)*

Grazie a questa tabella di conversione, puoi scartare completamente i titoli non riconosciuti e mappare ogni singola interazione di MovieLens direttamente sugli identificativi del tuo dump di TMDB.

2. Il Meccanismo: Item-to-Item Collaborative Filtering

Il Collaborative Filtering non richiede necessariamente di conoscere chi sia il tuo utente rispetto agli altri utenti. Si basa sull'approccio **Item-Based**: invece di chiedersi *"A quali utenti assomiglia il mio utente?"*, ci si chiede *"A quali film assomigliano i film visti dal mio utente, secondo i dati storici mondiali?"*.

Il processo si divide in due fasi distinte:

```
FASE OFFLINE (Una sola volta, su MovieLens)
[160.000 Utenti × 60.000 Film] ⟶ Calcolo Algoritmico ⟶ Matrice di Similarità Item-Item (S)
                                                        (Dimensioni: Film × Film)

FASE ONLINE (In tempo reale, sul tuo singolo utente)
[Cronologia Singolo Utente] × [Matrice S] ⟶ Punteggi di affinità per tutti i film non visti
```

Fase Offline: Calcolo della Matrice di Similarità (S)

Su MovieLens, due film sono considerati simili non se hanno la stessa trama, ma se **migliaia di utenti tendono ad apprezzarli entrambi**. Per calcolare questa correlazione tra tutti i film del catalogo, si usano solitamente due algoritmi:

- **Cosine Similarity / Adjusted Cosine:** Calcola l'angolo tra i vettori di voto di due film nello spazio degli utenti.
- **EASE (Embarrassingly Shallow Autoencoders):** Calcola una matrice simmetrica di pesi B dove la diagonale è azzerata. È la tecnica lineare più performante: trova la combinazione ottima di pesi per ricostruire la visione di un film basandosi sulla visione di tutti gli altri.

Il risultato di questa fase è una matrice statica: per ogni film A, hai una classifica dei film B, C, D statisticamente più affini secondo il pubblico globale.

Fase Online: Proiezione del Tuo Utente (*Folding-In*)

Quando il tuo singolo utente guarda dei film nel tuo sistema, non hai bisogno di ricalcolare nulla né di inserirlo in un cluster di utenti.

1. **Crei il vettore dell'utente:** Un vettore sparso dove ogni cella rappresenta un film di TMDB (es. 1 se visto/apprezzato, 0 se non visto).
2. **Moltiplicazione vettoriale:** Moltiplichi il vettore dell'utente per la matrice di similarità S.

$$\hat{R} = U \cdot S$$

3. **Punteggio aggregato:** Ogni film non visto riceve un punteggio dato dalla somma delle similarità con *tutti* i film che l'utente ha già visto.
4. **Filtro:** Escludi i film già guardati e ordini i restanti in ordine decrescente di punteggio.

3. Perché questo approccio è superiore ai soli metadati

Se usi soltanto le keyword di TMDB, il sistema consiglierà film con tag identici (es. *Alien* → *Aliens* → *Predator*, perché condividono *space*, *extraterrestrial*, *monster*).

Il Collaborative Filtering estratto da MovieLens scopre invece **correlazioni culturali e stilistiche invisibili ai tag**:

- Può scoprire che gli spettatori che amano *Blade Runner* spesso adorano anche *Her* o *Chinatown*, anche se uno è una storia d'amore fantascientifica e l'altro un noir poliziesco degli anni '70 ambientato a Los Angeles.
- I metadati descrivono il **soggetto** del film; i dati collaborativi catturano il **gusto** e la sensibilità del pubblico.

4. Come fonderlo con le tue Keyword TMDB (Pipeline Ibrida)

Il limite principale di MovieLens è il **Cold Start dei film recenti**: un film uscito lo scorso mese non è presente nei dati di MovieLens (avrà zero visualizzazioni storiche).

La soluzione ideale consiste nell'utilizzare un **punteggio ibrido**:

$$Punteggio\ Finale = \alpha \cdot Score_{MovieLens} + (1 - \alpha) \cdot Score_{Keyword\ TMDB}$$

- **Per i film di catalogo (presenti in MovieLens):** Lo score collaborativo guida la raccomandazione, garantendo suggerimenti sorprendenti e maturi.
- **Per i film nuovi (assenti in MovieLens):** Lo score collaborativo è 0, ma il film sale comunque in classifica grazie alla vicinanza semantica tra le sue keyword TMDB e le keyword dei titoli visti dall'utente.

User prompt: Due dubbi: funziona per serie e anime? Funziona per roba nuova(non nel db)? Questo csv a quando è aggiornato?

Response: Ecco le risposte dirette e puntuali ai tuoi tre dubbi.

1. Funziona per Serie TV e Anime?

- **Film Anime: Sì.** I lungometraggi anime distribuiti al cinema (*La città incantata*, *Akira*, *Your Name*, film di *Evangelion*, ecc.) sono ampiamente votati su MovieLens e mappati correttamente con gli ID film di TMDB.
- **Serie TV e Serie Anime: No, quasi per nulla.** MovieLens nasce e rimane un database per soli lungometraggi. I codici in `links.csv` rimandano esclusivamente agli ID film (`/movie/{id}`) di TMDB, non alle serie TV (`/tv/{id}`).
- **Come risolverlo se ti servono Anime e Serie TV:**
  - **Per gli Anime:** Esiste l'equivalente perfetto di MovieLens per l'animazione giapponese, ovvero i dump aperti di **MyAnimeList (MAL)** (disponibili su Kaggle e GitHub con oltre 300.000 utenti e milioni di voti su serie e OAV). Puoi mappare gli ID di MAL con TMDB via script usando il titolo.
  - **Per le Serie TV occidentali:** Non esiste un dataset open-source collaborativo massivo e pulito quanto MovieLens. Per le serie TV, l'approccio standard senza dati collaborativi propri è usare le **keyword, i creatori/showrunner e i generi di TMDB** tramite il metodo del centroide/profilo o tramite un Knowledge Graph.

2. Funziona per roba nuova (non presente nel DB)?

Con il solo modello collaborativo MovieLens: **No.** È il problema strutturale del **Cold Start**: se un film è uscito di recente o non è presente nei dati di addestramento, la sua riga nella matrice ha zero interazioni, quindi non riceverà mai un punteggio di affinità.

Come si risolve nella pratica: Il dump di TMDB si scarica aggiornato in tempo reale (TMDB rilascia file export giornalieri). Quando un titolo non ha riscontro collaborativo:

1. Il sistema verifica se il `tmdbId` esiste nella matrice MovieLens.
2. **Se esiste:** usa il punteggio collaborativo combinato con le keyword.
3. **Se non esiste (titolo recente):** il sistema commuta al 100% sul **Content-Based delle sole keyword TMDB**, confrontando i tag del nuovo titolo con il profilo/centroide dell'utente. In questo modo i film recenti entrano subito nelle raccomandazioni.

3. A quando è aggiornato questo CSV?

I dataset accademici di GroupLens (MovieLens) sono **snapshot storici statici**, non feed in tempo reale:

- **MovieLens 25M (`ml-25m`):** È congelato a **novembre 2019** (contiene circa 62.000 film).
- **MovieLens 32M (`ml-32m`):** È l'ultima release benchmark rilasciata da GroupLens, aggiornata a **fine 2023** (contiene circa 87.000 film e 32 milioni di recensioni).

Tutti i film usciti successivamente **non sono presenti** nei file di MovieLens.

Verdetto architetturale

Se il tuo catalogo include **film di catalogo storici**, MovieLens 32M ti dà un bootstrap collaborativo eccellente a costo zero.

Se invece il tuo catalogo include **serie TV, anime a episodi o uscite recenti**, l'infrastruttura vincente è un **sistema ibrido a due canali**:

1. **Canale Collaborativo (MovieLens 32M):** per i film pre-2024 presenti nel mapping.
2. **Canale Content-Based (Keyword TMDB):** per tutte le serie TV, le uscite recenti e come fallback generale.

User prompt: Ma se invece il dataset collaborativo venisse in qualche modo astratto? Ovvero, piuttosto che basarci su il singolo collegamento film-film, se noi arricchiamo i metadati del film introducendo le keyword budget, guadagni, punteggio, anno, lingue, e altri dati fondamentali come ad esempio la durata, noi possiamo collegare, ad esempio, i metadati di questo ad altri metadati di dell'altro film in modo che non è un film che punta ad un film, ma è una descrizione del film, esclusa ovviamente la descrizione testuale che punta ad un'altra descrizione e noi utilizziamo questo per addestrare un classificatore, un modello di regressione, ad esempio, o altro per predire, dato un film, dove si dovrebbe cercare, poi cerchiamo il candidato migliore, ovviamente, mostriamo tutto su uno spazio vettoriale denso e prendiamo i candidati più vicini. Che ne pensi?

Response: È un'intuizione architetturalmente brillante: stai formalizzando il passaggio da un modello **transduttivo** (vincolato agli ID rigidi dei film già visti) a un modello **induttivo** (basato sulle feature), realizzando quello che in letteratura si chiama **Knowledge Distillation da Collaborative Filtering a Content Space**.

Invece di usare MovieLens come un lookup-table statico, lo utilizzi come **supervisore per insegnare a uno spazio vettoriale continuo come si relazionano i metadati tra loro in base al gusto umano**.

Perché l'idea funziona (I Vantaggi)

1. **Risoluzione definitiva del Cold-Start:** Qualsiasi film uscito oggi, non appena inserito in TMDB con i suoi tag, può essere proiettato istantaneamente nello spazio vettoriale senza attendere che qualcuno lo voti.
2. **Generalizzazione a Serie TV e Anime:** Anche se MovieLens non ha mai visto una singola puntata di una serie o di un anime, se quell'anime ha le keyword *cyberpunk*, *dystopia*, *philosophical* e un punteggio medio alto, il modello lo collocherà nello stesso "quartiere" geometrico di film come *Blade Runner* o *Ghost in the Shell*.
3. **Decoppiamento dagli ID:** Non hai più bisogno di mantenere matrici giganti e sparse da 80.000 $\times$ 80.000 elementi; lavori con vettori densi compatti (es. 128 o 256 dimensioni) interrogabili con algoritmi di Nearest Neighbor (FAISS, HNSW).

Come formalizzare l'architettura (Il pattern Teacher-Student)

Il modo più solido e standard per implementare esattamente questa idea non è un classificatore generico, ma un processo a due stadi:

```
FASE 1: IL "TEACHER" (Spazio Collaborativo Puro)
MovieLens Matrix Factorization (ALS / SVD)
   ⟶ Estrae un vettore latente Z per ogni film noto (es. 128 dimensioni).
      Questo vettore cattura il "DNA socioculturale" del film.

FASE 2: LO "STUDENT" (Regressione / Projection Network)
Prende i Metadati TMDB (Keyword, Anno, Lingua, Score...)
   ⟶ Passa attraverso un Multi-Layer Perceptron (MLP) o Two-Tower
   ⟶ Obiettivo di addestramento: minimizzare la distanza (MSE o Cosine)
      tra il vettore predetto dai metadati e il vettore reale Z di MovieLens.

FASE 3: INFERENZA & INGESTION (Universale)
Nuovo Film / Serie TV / Anime
   ⟶ Estrai metadati TMDB ⟶ Passa nello Student ⟶ Vettore Denso
   ⟶ Inserimento nel Vector DB (Qdrant, FAISS)
```

In questo modo, lo *Student* impara relazioni implicite come: *"Il film con la combinazione di keyword A + B + C e punteggio > 8 tendono a finire in una regione dello spazio molto amata da chi guarda film d'autore"*.

Le insidie dei metadati: cosa funziona e cosa no

L'idea è ottima, ma la scelta dei metadati richiede attenzione per evitare bias distorti:

- **Budget e Incassi (Attenzione!):**
  - *Il problema:* Su TMDB, le **Serie TV non hanno quasi mai dati di budget e incassi al botteghino**, e i film indipendenti o internazionali hanno spesso valore 0.
  - *Il rischio:* Se il modello si abitua a usare budget e incassi come feature primarie per i film di Hollywood, fallirà miseramente quando gli passerai una serie TV o un anime. È preferibile **escludere o penalizzare fortemente budget/box office**, oppure normalizzarli come variabile binaria (es. *blockbuster / indie*).
- **Le Keyword (Il 70% del segnale):**
  - Rappresentano la feature più potente. Vanno codificate tramite vettori multi-hot pesati (TF-IDF/BM25 sulle keyword) o tramite piccoli embedding di parole, in modo che parole chiave rare e identitarie (*time-loop*, *heist*, *cyberpunk*) pesino molto più di tag generici (*sequel*, *duringcreditsstinger*).
- **Punteggio e Numero Voti (Il filtro qualità):**
  - Usare la **media bayesiana ponderata** di TMDB (che unisce `vote_average` e `vote_count`). Questo permette al modello di capire se il titolo è un capolavoro di culto o un b-movie, posizionandolo nella corretta fascia qualitativa.
- **Anno di uscita e Durata (Feature continue di contesto):**
  - L'anno cattura l'estetica e la tecnologia del periodo (gli anni '80 hanno un'atmosfera diversa dagli anni 2020). La durata serve a distinguere corti, lungometraggi standard ed epopee da oltre 3 ore (per le serie TV si può usare la durata media dell'episodio).
- **Lingua originale e Paese di produzione (Feature categoriche):**
  - Fondamentali per riconoscere cluster naturali (cinema coreano, anime giapponesi, produzioni nordeuropee).

Verdetto

L'idea non solo è valida, ma è esattamente la soluzione industriale per superare i limiti di MovieLens: **usi i dati collaborativi per addestrare la bussola, ma navighi il catalogo usando unicamente la mappa dei metadati.**

In questo modo, dato un film visto dal tuo utente, proietti le sue coordinate nello spazio latente e interroghi il database per trovare i candidati più vicini per distanza coseno, estendendo la ricerca a qualsiasi film, serie o anime presente in TMDB.

User prompt: Ok, quindi l'idea è di creare questo classificatore. Precalcolare per ogni film e per ogni serie il suo vettore, quindi ehm rappresentare in maniera densa ogni film ed ogni serie nello spazio e poi creare i link utilizzando il classificatore, quindi per ogni film collegarlo ad altri film. In questo modo creeremmo un grafo, un grafo che è fortemente connesso perché ogni film punta ad altri punti nello spazio, quindi si prenderà sicuramente il uno o più film vicini, quindi ogni film avrà i suoi candidati simili e possiamo creare dei dei cluster, ad esempio, utilizzando Leiden e poi restituire quelli come suggerimenti. Per un film restituire il un cluster o comunque una serie di cluster e poi in base al match col con lo storico dell'utente o comunque con le sue preferenze, restituire il cluster più adeguato, i cluster più adeguati. Anche perché considera che poi lui avrà un sacco di film visti o comunque anche se ne ha pochi, ne avrà di film visti e quindi possiamo utilizzare questo meccanismo per fare la la prima cernita dei primi magari 500, 1000 film da raccomandare, poi applicare punteggi, re-ranking e altre cose nel restituire il migliore. Mi sono perso qualche dettaglio, qualche cosa utile che non sto considerando o altro.

Response: L'architettura che hai delineato è concettualmente solida e ricalca logiche industriali avanzate: **proiezione dei metadati in uno spazio denso** $\rightarrow$ **k-NN graph di affinità** $\rightarrow$ **community detection (Leiden)** $\rightarrow$ **retrieval per cluster** $\rightarrow$ **re-ranking**.

Ci sono però **5 dettagli architetturali critici** e alcune insidie matematiche che rischi di sottovalutare se non li progetti a monte:

1. Il problema del "Centroide Unico" (Multi-Interest dell'utente)

Se un utente ha visto 50 film, l'errore più comune è fare la media algebrica di tutti i vettori per creare un "vettore utente" e cercare il cluster corrispondente.

- **Il rischio:** Se l'utente guarda *cartoni animati per famiglie* il pomeriggio e *thriller psicologici violenti* la notte, la media dei due vettori cade in uno spazio geometrico nullo (ad esempio film d'avventura mediocri) che non rappresenta nessuno dei due gusti.
- **La correzione:** Non assegnare l'utente a un solo cluster. Fai un mini-clustering (o conteggio di frequenza) **sullo storico dell'utente**:
  - Scoprirai che l'utente ha visto film che ricadono per il 60% nel *Cluster A (Sci-fi dark)* e per il 40% nel *Cluster B (Commedie stand-up)*.
  - Il retrieval dei 500–1000 film deve essere **proporzionale** (pescaggio di quote dai rispettivi cluster) per rispettare i molteplici gusti della persona.

2. L'Hubness Problem nel Grafo k-NN

Quando crei archi collegando ogni film ai suoi k vicini più prossimi nello spazio denso:

- **Il fenomeno:** Negli spazi ad alta dimensionalità alcuni nodi (spesso film molto famosi o con metadati mediamente diffusi) diventano "hub". Cioè, compaiono tra i vicini di quasi tutti gli altri film.
- **La conseguenza:** Se non bilanci il grafo, i cluster di Leiden risulteranno sbilanciati: avrai 2–3 "mega-cluster mostruosi" con migliaia di nodi interconnessi da questi hub, e una marea di micro-isole di nicchia disconnesse.

- **La correzione:**
  - Usa un **Mutual k-NN** (un arco esiste solo se A è vicino a B **E** B è vicino a A) oppure normalizza il grafo usando la metrica di similarità di Jaccard/Cosina simmetrizzata.
  - Taratura del parametro di **risoluzione (γ)** in Leiden: ti servirà una risoluzione medio-alta per forzare l'algoritmo a trovare **micro-generi compatti** (es. 200–300 cluster da 50–150 film) anziché 10 macro-cluster generici.

3. Vettori vs Grafo: A cosa serve davvero Leiden?

C'è una ridondanza operativa da tenere presente:

- Se hai già i vettori densi di tutti i titoli, **un Vector Search Engine (come FAISS o Qdrant basati su indice HNSW) fa già il retrieval dei 500 candidati migliori in meno di 5 millisecondi**, senza bisogno di passare per il grafo.
- **Perché allora Leiden ha un valore immenso?** Non per il retrieval grezzo, ma per la **struttura a caroselli (UI) e la diversificazione**:
  - Un feed Netflix non mostra una classifica piatta da 1 a 500, ma righe tematiche (*"Perché hai visto X"*, *"Cyberpunk e futuri distopici"*, *"Cinema d'autore orientale"*).
  - I cluster di Leiden estraggono esattamente questi micro-generi latenti senza doverli etichettare a mano. Puoi quindi usare il cluster per formare i **caroselli tematici** e per garantire che nei 500 candidati ci sia diversità e non 500 cloni dello stesso titolo.

4. Il vincolo del "Partizionamento Rigido" (Hard Clustering)

Leiden assegna per definizione ogni film a **un solo cluster** (partizione disgiunta).

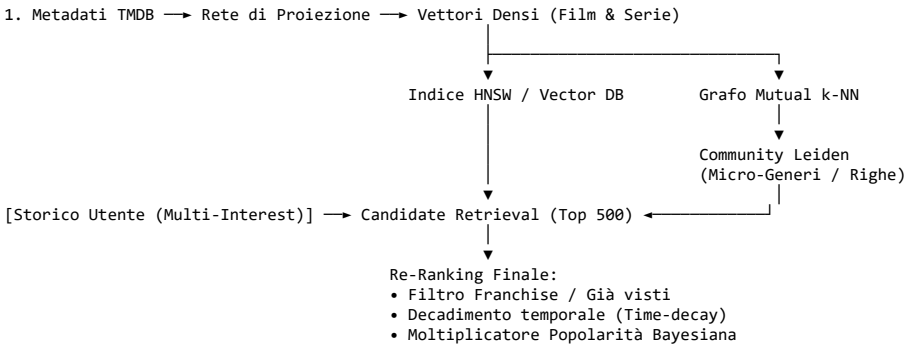
- **Il limite:** Molte opere sono intrinsecamente ibride (es. *Alien* è 50% Sci-Fi e 50% Body Horror; *Spider-Man: Un nuovo universo* è animazione, cinecomic e commedia contemporaneamente).
- **La correzione:** Se forzi un film di confine in un solo cluster, rischi di renderlo invisibile agli utenti interessati all'altra sua metà. Quando calcoli i vicini, conserva per ogni titolo i **pesi di appartenenza** (es. probabilità o distanza dal centroide del cluster) verso i primi 2 o 3 cluster più affini, non solo quello dominante.

5. Logica di Franchise e Sequenze nel Re-Ranking

Nel passaggio dai 500 candidati al ranking finale, il grafo di similarità da solo farà un errore sistematico:

- Se un utente ha visto *Il Signore degli Anelli: La Compagnia dell'Anello*, i candidati matematicamente più vicini in assoluto saranno *Le Due Torri* e *Il Ritorno del Re*.
- **Nel re-ranking devi gestire le saghe:**
  - Se l'utente ha già visto il capitolo 2 e 3, devono essere esclusi immediatamente.
  - Se non li ha visti, **non mostrare tutti i sequel insieme nello stesso blocco**: consiglia solo il capitolo cronologicamente successivo (*Next-in-Sequence*), penalizzando gli altri per evitare di cannibalizzare lo spazio visivo.
  - **Per le Serie TV:** Il sistema deve consigliare la serie nel suo complesso o la stagione successiva, evitando di trattare singoli episodi o stagioni come entità slegate nel grafo.

Riepilogo del Flusso Ottimizzato



User prompt: Sì, ok, ha senso, soprattutto al discorso dei microcluster o comunque dell'assegnazione di di vettori multipli per ogni film, anche per lo storico dell'utente, di classificare i film in base ai cluster, che comunque ora ci verrà facile, avendo questo questo grafo, ci verrà facile clusterizzarli e in base ai cluster, per ogni cluster dare dei suggerimenti e magari in base al tipo di profilo, in base alla visione recente, prediligere un cluster rispetto ad un altro o vedere tra questi cluster quelli che sono più simili e che prevalgono e trascurare gli altri, comunque, ogni cluster diventa una riga di suggerimenti.

Response: Questo schema corrisponde esattamente all'architettura di **Page Generation / Two-Stage Slate Recommendation** utilizzata dalle principali piattaforme: il cluster non è solo un raggruppamento matematico nel backend, ma diventa l'**unità atomica dell'interfaccia utente** (il singolo carosello orizzontale).

Il flusso operativo per rendere questa logica autonoma e performante si articola in quattro componenti chiave.

1. Etichettatura Automatica delle Righe (Auto-Naming)

Se ogni cluster di Leiden diventa una riga, il sistema deve sapere che titolo dare al carosello senza doverlo inserire manualmente per centinaia di gruppi.

- **Come estrarre il titolo:** Per ogni cluster, calcola il punteggio **TF-IDF o BM25 delle keyword TMDB** appartenenti ai film di quel gruppo.
  - Il termine *Term Frequency* (TF) misura quanto spesso una keyword compare nel cluster.
  - Il termine *Inverse Document Frequency* (IDF) penalizza le keyword generiche presenti ovunque nel catalogo (es. *drama*, *sequel*).
- **Il risultato:** Le 2 o 3 keyword con il punteggio più alto diventano l'etichetta del carosello (es. cluster con *time travel* + *dystopia* $\rightarrow$ carosello: *"Viaggi nel tempo e futuri distopici"*).

2. Calcolo del Peso dei Cluster per l'Utente (Row Ranking)

L'utente non ha un singolo profilo piatto, ma una **distribuzione di probabilità sui cluster**. Per decidere quali caroselli mostrare in cima alla pagina:

1. **Allocazione delle visioni:** Ogni film visto dall'utente accende il punteggio del relativo cluster (o distribuisce frazioni di punto sui 2 cluster più vicini al film).

2. **Decadimento Temporale Esponenziale ($Half\text{-}Life$):** Applica un dimezzamento progressivo del peso (es. ogni 14 o 30 giorni).
    - Se l'utente ha visto 40 film d'animazione l'anno scorso ma negli ultimi 3 giorni ha guardato 2 horror psicologici, il cluster *Horror* riceve un moltiplicatore di freschezza temporale che lo proietta immediatamente nelle prime 3 righe dell'interfaccia.
3. **Ordinamento delle Righe:** Le prime 4–6 righe a schermo corrispondono ai cluster con il punteggio complessivo più alto, riflettendo la combinazione tra preferenze storiche consolidate e mood recente.

3. Ordinamento dei Titoli Dentro il Carosello (Item Ranking)

Una volta scelti i cluster da mostrare, come si ordinano i 15–20 titoli da sinistra a destra all'interno della riga?

- **Distanza dal Centroide del Cluster:** I titoli al centro geometrico del cluster rappresentano gli esempi più puri e riconoscibili di quel micro-genere.
- **Moltiplicatore Bayesiano di Qualità:** I film con alto rapporto tra voto medio e numero di valutazioni su TMDB vengono spinti verso sinistra (le prime posizioni visibili a schermo).
- **Filtro Saghe (Anti-Cannibalizzazione):** Se il cluster include la trilogia del *Cavaliere Oscuro*, mostra solo il primo capitolo non ancora visto dall'utente. Gli altri capitoli compaiono solo dopo che il precedente è stato contrassegnato come completato.

4. Esplorazione e Prevenzione della "Filter Bubble"

Se il sistema mostra sempre e solo i cluster con il punteggio più alto, l'utente finisce in un loop monotematico (se guarda 3 thriller, vede solo thriller all'infinito).

- **La regola 80/20 per la griglia:**
    - **80% delle righe:** Cluster ad alta affinità (gusti consolidati + visioni recenti).
    - **20% delle righe:** Cluster adiacenti o di scoperta.
- **Come trovare cluster adiacenti:** Calcola la distanza del coseno tra i centroidi dei cluster nello spazio denso. Se l'utente ama il cluster *Cyberpunk Noir*, il sistema può proporre a metà pagina il cluster adiacente *Techno-Thriller Coreano*, creando serendipità coerente con i suoi gusti.
- **Negative Feedback implicito:** Se un carosello viene mostrato per 3 sessioni consecutive senza che l'utente clicchi su nessun titolo al suo interno, il punteggio di quel cluster viene temporaneamente depresso per lasciare spazio ad altri generi.
